Table 1. Lean notation and tactics used in the Lean listings of the paper.

Lean	Meaning
`import Mathlib`	load the Mathlib library
`example`, `theorem t ... : P := by`	unnamed or named statement P; `:= by` opens the proof in tactic mode
`def`, `abbrev`	definition (`abbrev`: unfolded automatically)
`@[ext] structure`, θ`.a`	record with named fields (equal if its fields are); field `a` of θ
`variable {`Θ `: Type}`	makes Θ an implicit argument
`h : `ρ `< 1`	hypothesis named `h` stating $\rho < 1$
$\mathbb{N}$, $\mathbb{Z}$, $\mathbb{R}$, `Prop`	natural numbers, integers, reals, propositions
`Fin n`, `i.val`	indices $\{0, \dots, n-1\}$; the natural number i
`f x y`, `fun x => e`	function application $f(x, y)$; the function $x \mapsto e$
$\vdash$ `P`	the goal: P remains to be proved
$\forall$ `x, P x`	for all x, $P(x)$
`P` $\to$ `Q`, `P` $\leftrightarrow$ `Q`	implication, equivalence
`a` $\neq$ `b`	$a \neq b$, i.e. `a = b` $\to$ `False`
`|x|`, $\sum$ `i, f i`	absolute value; sum over all indices i
`| 0 => ..., | k + 1 => ...`	definition by recursion on k
`thm a h`	theorem `thm` applied to `a` and a proof `h`
`(by tac)`	a proof term found by the tactic `tac`
`sorry`	placeholder that closes any goal; Lean warns
`#print axioms t`	the axioms that `t` depends on
`intro x h`	"let x be given and assume `h`" (for $\forall$, $\to$; for $a \neq b$ it assumes $a = b$)
`constructor`, $\cdot$	split $P \leftrightarrow Q$ into two implications; one bullet per goal
`have h : P := ...`	prove an intermediate claim P
`exact t`	the term `t` proves the goal
`rw [h]`	rewrite with the equation or equivalence `h`; `rw [traj]` unfolds `traj`
`calc a` $\leq$ `b := p1, _` $\leq$ `c := p2`	chain of (in)equalities, one proof per step
`linarith`	linear arithmetic over the hypotheses
`simp [h]`, `simpa [h] using e`	simplify (also with `h`); simplify `e` and the goal, close it with `e`
`norm_num at h`	evaluate numerals in `h`; closes the goal if false
`field_simp`	clear denominators, using hypotheses such as $\varphi_1 \neq 0$
`induction k with | zero => ... | succ k ih => ...`	induction on k, with induction hypothesis `ih`
`ext i`	equal records or functions, component by component

We refer to (de Moura and Ullrich, 2021) for more detailed explanations.

REFERENCES

de Moura, L. and Ullrich, S. (2021). The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, 625–635. Springer.